PHYB57 · CHAPTER 19 · 第 19 章

求根 · 全面复习讲义

Root Finding · Comprehensive Review Notes

中英双语 · KaTeX 公式 · 深色阅读版

Bilingual · KaTeX formulas · dark reading edition

KaTeX 公式 · KaTeX | 中英对照 · Bilingual | 每页考点速记 · Key points on every page | 按幻灯片顺序 · Slide order | 拓展 + 6 道练习 · Extensions + 6 problems

覆盖 C19 全部 10 页幻灯片：求根问题与 fsolve 的成功/失败、容差与误差度量、$1/x$ 的"假收敛"、二分法（原理、算法、代码、例子与局限）、Newton-Raphson 法（推导、代码与局限）、用 fsolve 求多项式的根。

> Covers all 10 slides of C19: the root-finding problem and fsolve's success and failure, tolerance and error metrics, the false convergence of $1/x$, the bisection method (idea, algorithm, code, example, limits), the Newton-Raphson method (derivation, code, limits), and roots of a polynomial with fsolve.

另加（标 拓展 EXT）：二分法步数公式、线性与二次收敛的数值对比、Newton 失败的三种典型情形、fsolve 初值与根的对应、SciPy 推荐写法；最后是公式总表和 6 道完整解答的练习题。所有数值都用 Python 复算过；幻灯片第 10 页漏掉的根 $x = -1$ 已标出。

> Added (tagged 拓展 EXT): the bisection step-count formula, a numerical comparison of linear and quadratic convergence, three typical Newton failures, how fsolve's initial guess picks the root, and recommended SciPy calls; then a formula sheet and 6 fully worked problems. Every number was recomputed in Python; the root $x = -1$ missing from slide 10 is flagged.

PHYB57 复习用 · 2026 年 11 月 5 日

> For PHYB57 review · 5 November 2026

考点速记 KEY POINTS

- 根：$f(x_r) = 0$。
  Root: $f(x_r) = 0$.
- Newton：$x_1 = x_0 - f(x_0)/f'(x_0)$，二次收敛。
  Newton: $x_1 = x_0 - f(x_0)/f'(x_0)$, quadratic.
- 二分法：$f(a)f(b) < 0$，线性收敛，必收敛。
  Bisection: $f(a)f(b) < 0$, linear, always converges.
- $|f|$ 小 ≠ 找到根（$1/x$）。
  Small $|f|$ ≠ a root ($1/x$).

本章地图与记号

Chapter map and notation

节 Sec.	内容 Content	幻灯片 Slides
1	求根问题与 fsolve 的成功/失败 · The problem; fsolve success and failure	1–2
2	容差与误差度量；$1/x$ 的例子 · Tolerance and error metrics; the $1/x$ example	3–4
3	二分法：原理、算法、例子、局限 · Bisection: idea, algorithm, example, limits +步数 step count	5–7
4	Newton-Raphson 法：推导、实现、局限 · Newton-Raphson: derivation, code, limits +失败情形 failures	8–9
5	用 fsolve 求多项式的根 · Polynomial roots with fsolve +初值 initial guesses	10
—	公式总表 · 练习 Q1–Q6 · 答案与来源 · Formula sheet · Problems · Key & sources	—

记号约定 / Notation

根与迭代序列 / root and iterates

$$f(x_r) = 0, \qquad x_0,\ x_1,\ x_2,\ \ldots \to x_r$$

误差 / error

$$e_k = |x_k - x_r| \qquad (\text{usually unknown: } x_r \text{ is what we seek})$$

- 容差（tolerance, tol）：可接受的误差水平；误差小于 tol 即视为收敛。
  > Tolerance (tol): the acceptable error level; an error below tol counts as converged.
- 线性收敛：$e_{k+1} \approx c\,e_k$ $(c < 1)$；二次收敛：$e_{k+1} \approx C\,e_k^2$，有效数字每步约翻倍。
  > Linear convergence: $e_{k+1} \approx c\,e_k$ $(c < 1)$; quadratic convergence: $e_{k+1} \approx C\,e_k^2$, correct digits roughly double each step.
- 括根区间（bracket）：$[a, b]$ 满足 $f(a)\,f(b) < 0$。
  > Bracket: an interval $[a, b]$ with $f(a)\,f(b) < 0$.
- 标 拓展 EXT 的内容超出幻灯片。
  > Items tagged 拓展 EXT go beyond the slides.

考点速记 KEY POINTS

- 真误差 $|x_k - x_r|$ 通常无法直接算。
  The true error $|x_k - x_r|$ is usually unavailable.
- 括根：$f(a)f(b) < 0$。
  Bracket: $f(a)f(b) < 0$.
- 线性：误差按比例降；二次：位数翻倍。
  Linear: error shrinks by a factor; quadratic: digits double.
- 收敛 = 误差度量 < tol。
  Converged = error metric < tol.

1 求根问题

1 Root finding problem statement

定义 / Definition

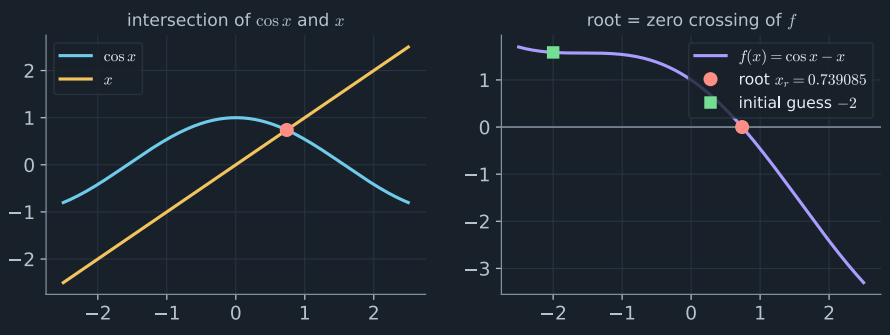
- 函数 $f(x)$ 的根或零点是满足 $f(x_r) = 0$ 的值 x_r。
  > A root or zero of a function $f(x)$ is a value x_r such that $f(x_r) = 0$.
- 有些函数（如二次函数）的根一眼可得：$f(x) = x^2 - 9$ 的根是 $x = \pm 3$。
  > Some functions, like quadratics, have easily identified roots: $f(x) = x^2 - 9$ has roots $x = \pm 3$.

数值近似 / Numerical approximation

对精确解难以求出的复杂函数，可以用数值方法，例如 SciPy 库的 fsolve。例：求 $\cos x = x$，即 $f(x) = \cos x - x$ 的根，初始猜测 -2：

> For more complex functions where exact solutions are hard to find, numerical methods such as SciPy's fsolve can be used. Example: solve $\cos x = x$, i.e. find the root of $f(x) = \cos x - x$, from the initial guess -2:

```
from scipy import optimize
f = lambda x: np.cos(x) - x
r = optimize.fsolve(f, -2)
```



$\cos x = x$ 没有解析解；根 $x_r = 0.739085$ 是两曲线交点

/ $\cos x = x$ has no closed-form solution; the root $x_r = 0.739085$ is where the curves cross

这种方法给出的是近似根，并且有局限，例如可能达到函数调用次数上限。

> This approach finds approximate roots, with limitations such as reaching a maximum number of function evaluations.

局限 / Limitations

- 若达到最大迭代次数仍未收敛，数值方法可能失败。
  > Numerical methods may fail if they reach the maximum number of iterations without converging.
- 理解这些方法的约束，是在工程与科学中正确使用它们的关键。
  > Understanding these constraints is crucial for applying them properly in engineering and science.

考点速记 KEY POINTS

- 求根 = 解 $f(x) = 0$；$g(x) = h(x)$ 改写为 $g - h = 0$。
  Root finding solves $f(x) = 0$; rewrite $g = h$ as $g - h = 0$.
- $\cos x = x$ 的根 ≈ 0.739085。
  Root of $\cos x = x$: ≈ 0.739085.
- fsolve(f, x0)：需要函数和初始猜测。
  fsolve(f, x0): needs a function and a guess.
- 达到最大调用次数 = 失败信号。
  Hitting the call limit signals failure.

1.2 fsolve 的成功与失败

1.2 fsolve: success and failure

成功 / Success

```
import numpy as np
from scipy import optimize

f = lambda x: np.cos(x) - x
r = optimize.fsolve(f, -2)
print("r =", r)

# Verify the solution is a root
result = f(r)
print("result=", result)
```

```
r = [0.73908513]
result= [0.]
```

返回 $r = 0.73908513$，代回 $f(r) = 0$：这是真正的根。

> It returns $r = 0.73908513$ and $f(r) = 0$ on substitution: a genuine root.

失败 / Failure

```
f = lambda x: 1/x

r, infodict, ier, mesg = optimize.fsolve(
        f, -2, full_output=True)
print("r =", r)

result = f(r)
print("result=", result)

print(mesg)
```

```
r = [-3.52047359e+83]
result= [-2.84052692e-84]
The number of calls to function
  has reached maxfev = 400.
```

$1/x$ 没有根。fsolve 把 x 推到 -3.5×10^{83}，$f(r) \approx -2.8 \times 10^{-84}$ 看起来"几乎为零"，但消息说明调用次数已达上限 400：求解失败。

> $1/x$ has no root. fsolve drives x to -3.5×10^{83}, where $f(r) \approx -2.8 \times 10^{-84}$ looks "almost zero", but the message says the 400-call limit was hit: the solve failed.

full_output 的四个返回值 拓展 EXT / The four outputs of full_output

名称 Name	含义 Meaning
r	解的估计（数组）· the solution estimate (an array)
infodict	字典：nfev（调用次数，本例 402）、fvec（$f(r)$）等 · dictionary: nfev (calls, 402 here), fvec ($f(r)$), …
ier	**ier == 1 才表示成功**；本例 ier = 2 · **only ier == 1 means success**; here ier = 2
mesg	失败原因的文字说明 · a text explanation of the failure

注意 / Caution

只看 $f(r) \approx 0$ 会把失败误认成成功。可靠的检查是三步：看 ier（或 mesg）、代回 $f(r)$、并检查 r 本身是否合理（10^{83} 显然不是）。

> Looking only at $f(r) \approx 0$ mistakes this failure for success. A reliable check has three steps: read ier (or mesg), substitute $f(r)$, and ask whether r itself is plausible (10^{83} clearly is not).

考点速记 KEY POINTS

- full_output=True 返回 r, infodict, ier, mesg。
  full_output=True returns r, infodict, ier, mesg.
- $f(r) \approx 0$ 不足以证明是根。
  $f(r) \approx 0$ alone does not prove a root.
- ier == 1 才是成功。
  Only ier == 1 is success.
- maxfev = 400：函数调用上限。
  maxfev = 400: the function-call limit.

2 数值方法中的容差

2 The concept of tolerance in numerical methods

容差的定义 / Definition of tolerance

- 容差指数值解中可接受的误差水平。
  > Tolerance is the acceptable level of error in a numerical solution.
- 若误差小于设定的容差，就认为解已收敛。
  > A solution is considered converged if its error is smaller than the defined tolerance.

误差度量 / Error measurement

1 残差 / residual

$$|f(x)| < \text{tol}$$

函数值离 0 有多近

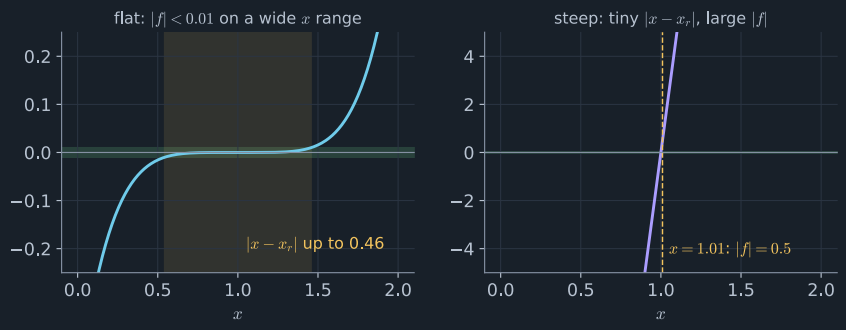
/ how close the function value is to zero

2 步长 / step size

$$|x_{i+1} - x_i| < \text{tol}$$

相邻两次猜测之差，反映收敛速度

/ difference of successive guesses; shows convergence speed



两种度量可能给出相反的结论（拓展） / the two metrics can disagree (extension)

左：$f = \frac{1}{2}(x-1)^5$ 在根附近很平，$|f| < 0.01$ 的区间宽达 ± 0.46，残差小但 x 可能差得远。右：$f = 50(x-1)$ 很陡，$x = 1.01$ 已很接近根，残差却是 0.5。

> Left: $f = \frac{1}{2}(x-1)^5$ is flat near its root, so $|f| < 0.01$ holds across ± 0.46: small residual, yet x may be far off. Right: $f = 50(x-1)$ is steep, so $x = 1.01$ is already close to the root but the residual is 0.5.

在应用中的重要性 / Importance in applications

- 不同应用需要不同精度，容差要按具体需求设定。
  > Different applications need different precision, so tolerances must be set to the specific need.
- 谨慎选择容差，才能避免假收敛或过早停止。
  > Careful choice of tolerance avoids false convergence or premature stopping.

考点速记 KEY POINTS

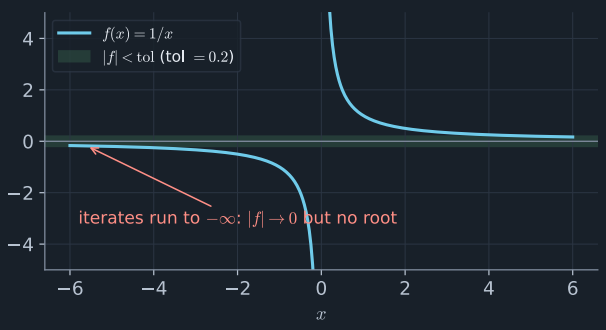
- 两种度量：$|f(x)|$ 与 $|x_{i+1} - x_i|$。
  Two metrics: $|f(x)|$ and $|x_{i+1} - x_i|$.
- 陡峭函数：x 准但残差可能大。
  Steep f: accurate x, possibly large residual.
- 平坦函数：残差小 $\neq x$ 准。
  Flat f: small residual ≠ accurate x.
- 容差按应用需要设定。
  Set tol to the application's needs.

2.2 容差与求根的实例

2.2 Practical example of tolerance and root finding

例子分析 / Example analysis

- 考虑函数 $f(x) = 1/x$。
  > Consider the function $f(x) = 1/x$.
- 若以 $|f(x)|$ 为误差度量并设一个小容差，即使根根本不存在，也可能显示"已收敛"。
  > With $|f(x)|$ as the error metric and a small tolerance, the method may report convergence even though no root exists.
- 这说明正确解读容差和误差度量的重要性。
  > This shows the importance of correctly interpreting tolerance levels and error metrics.



$$|f(x)| = \frac{1}{|x|} < \text{tol} \iff |x| > \frac{1}{\text{tol}} \qquad \text{e.g. tol} = 10^{-6}:\ x = -2 \times 10^{6}\ \text{"passes"}$$

$|f|$ 小只是因为 x 很大，不是因为接近根 / $|f|$ is small only because x is huge, not because a root is near

容差的使用 / Use of tolerance

- 合适的容差设置能让求根方法表现最佳，并在可接受的误差范围内给出有意义的解。
  > An appropriate tolerance lets the root-finding method perform well and give a meaningful solution within acceptable error bounds.
- 始终同时检查误差大小和算法行为来验证结果。
  > Always validate results by checking both the error magnitude and the algorithm's behaviour.

要点 · 验证清单 拓展 EXT / **Key takeaway · validation checklist**

在数值分析中，容差不只是技术参数，它直接决定结果的可靠性和准确性。验证清单：① 解是否在合理范围；② 步长 $|x_{i+1} - x_i|$ 是否也在减小；③ 求解器是否报告成功（ier、converged）；④ 若能括根，f 在两侧是否变号。

> In numerical analysis, tolerance is not just a technical parameter; it decides how reliable and accurate the results are. Checklist: ① is the solution in a sensible range; ② is the step $|x_{i+1} - x_i|$ shrinking too; ③ does the solver report success (ier, converged); ④ if a bracket exists, does f change sign across it.

考点速记 KEY POINTS

- $1/x$ 无根，但 $|x| \to \infty$ 时 $|f| \to 0$。
  $1/x$ has no root, yet $|f| \to 0$ as $|x| \to \infty$.
- 只用 $|f| <$ tol 会假收敛。
  Using only $|f| <$ tol gives false convergence.
- 同时看误差和算法行为。
  Check both error and algorithm behaviour.
- 变号检查能识别真根。
  A sign-change check confirms a real root.

3 二分法
3 Bisection method

定义 / Definition

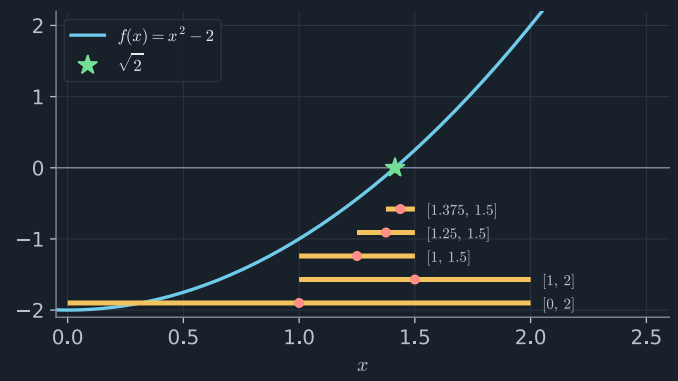
- 二分法是应用介值定理（Intermediate Value Theorem）的求根方法。
  > The bisection method is a root-finding technique that applies the Intermediate Value Theorem.
- 用于求连续函数 $f(x)$ 在变号区间 $[a,b]$ 上的根。
  > It finds a root of a continuous function $f(x)$ on an interval $[a,b]$ where the function changes sign.

$$f \text{ continuous on } [a,b], \quad f(a)\cdot f(b)<0 \quad \Longrightarrow \quad \exists\, c\in(a,b):\ f(c)=0$$

至少一个根；可能不止一个 / at least one root; there may be more

过程 / Process

- 计算中点 $m=(a+b)/2$。
  > Calculate the midpoint $m=(a+b)/2$.
- 计算 $f(m)$，根据 $f(m)$ 的符号更新左端 a 或右端 b（保留仍然变号的那一半）。
  > Evaluate $f(m)$; depending on its sign, update the left bound a or the right bound b (keep the half that still changes sign).
- 重复，直到误差或容差足够小。
  > Repeat until the error or tolerance is small enough.



x^2-2 在 $[0,2]$ 上：每一行是一次迭代后的区间，长度逐次减半，红点为中点

/ x^2-2 on $[0,2]$: each bar is the interval after one step, halving each time; red dots are midpoints

需要多少步 拓展 EXT / How many steps

$$|m_n - x_r| \le \frac{b-a}{2^n} \quad \Rightarrow \quad n \ge \log_2 \frac{b-a}{\varepsilon} \ \text{ guarantees } |m_n-x_r|\le\varepsilon$$

m_n 为第 n 个中点；步数只取决于区间长度和精度，与 f 无关

/ m_n is the n-th midpoint; the count depends only on the width and accuracy, not on f

考点速记 KEY POINTS

- 前提：f 连续且 $f(a)f(b)<0$。
  Needs continuous f and $f(a)f(b)<0$.
- 每步误差界减半。
  The error bound halves each step.
- 保留变号的那一半。
  Keep the half with the sign change.
- 步数 $n \ge \log_2 \frac{b-a}{\varepsilon}$。
  Steps $n \ge \log_2 \frac{b-a}{\varepsilon}$.

3.2 算法与实现

3.2 Algorithm and implementation

算法步骤 / Algorithm outline

1. 检查区间：确认 $f(a)$ 与 $f(b)$ 异号。
   > Check bounds: ensure $f(a)$ and $f(b)$ have opposite signs.
2. 算中点：$m = (a+b)/2$。
   > Calculate midpoint: $m = (a+b)/2$.
3. 判断中点：若 $|f(m)| <$ tol，m 即为根；若 $f(a)$ 与 $f(m)$ 异号，区间更新为 $[a, m]$；若 $f(m)$ 与 $f(b)$ 异号，更新为 $[m, b]$。
   > Evaluate midpoint: if $|f(m)| <$ tol, m is the root; if $f(a)$ and $f(m)$ have opposite signs, update to $[a, m]$; if $f(m)$ and $f(b)$ do, update to $[m, b]$.
4. 重复：直到达到所需精度。
   > Repeat: continue until the desired accuracy is reached.

读代码 / Reading the code

代码用"同号"判断：sign(f(a)) == sign(f(m)) 说明根不在 $[a, m]$，于是令 $a = m$。这与上面"异号就保留"是同一件事的两种说法。函数是递归的：每次调用自己，区间减半；停止条件是 $|f(m)| <$ tol。

> The code tests for the same sign: sign(f(a)) == sign(f(m)) means the root is not in $[a, m]$, so set $a = m$. This is the "keep the opposite-sign half" rule stated the other way round. The function is recursive: each call halves the interval; it stops when $|f(m)| <$ tol.

```python
import numpy as np

def my_bisection(f, a, b, tol):
    # approximates a root, R, of f bounded
    # by a and b to within tolerance
    # | f(m) | < tol with m the midpoint
    # between a and b Recursive implementation

    # check if a and b bound a root
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
         "The scalars a and b do not bound a root")

    # get midpoint
    m = (a + b)/2

    if np.abs(f(m)) < tol:
        # stopping condition, report m as root
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        # case where m is an improvement on a.
        # Make recursive call with a = m
        return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
        # case where m is an improvement on b.
        # Make recursive call with b = m
        return my_bisection(f, a, m, tol)
```

注意 拓展 EXT / Caution

这个实现只用 $|f(m)| <$ tol 停止。对很平的函数（上一节左图）它可能停得太早；对舍入误差使 $|f|$ 永远达不到 tol 的情形，递归会一直进行直到 Python 的递归深度上限（RecursionError）。实用写法通常加上区间宽度条件 $(b-a)/2 <$ xtol 和最大步数。

> This implementation stops only on $|f(m)| <$ tol. For a very flat function (left panel of the previous section) it may stop too early; if round-off keeps $|f|$ above tol, the recursion runs until Python's recursion limit (RecursionError). Practical versions also stop on interval width $(b-a)/2 <$ xtol and cap the number of steps.

考点速记 KEY POINTS

- 先检查 $f(a)$、$f(b)$ 异号，否则报错。
  First check opposite signs, else raise.
- 递归停止条件：$|f(m)| <$ tol。
  Recursion stops at $|f(m)| <$ tol.
- 同号 → 用 m 替换那一端。
  Same sign → replace that end with m.
- 实用版加宽度条件和步数上限。
  Practical code adds a width test and step cap.

3.3 例子与局限

3.3 Example and limitations

例：$f(x) = x^2 - 2$，初始区间 $[0, 2]$。容差 0.1 时得 $r_1 = 1.4375$；容差 0.01 时得 $r_{01} = 1.4140625$，更接近真根 $\sqrt{2} = 1.41421\ldots$

> Example: $f(x) = x^2 - 2$, initial interval $[0, 2]$. With tolerance 0.1 the method finds $r_1 = 1.4375$; with 0.01 it finds $r_{01} = 1.4140625$, closer to the true root $\sqrt{2} = 1.41421\ldots$

步 Step	a	b	m	$f(m)$	更新 Update
1	0	2	1	-1	$a \leftarrow m$
2	1	2	1.5	$+0.25$	$b \leftarrow m$
3	1	1.5	1.25	-0.4375	$a \leftarrow m$
4	1.25	1.5	1.375	-0.109375	$a \leftarrow m$
5	1.375	1.5	**1.4375**	$+0.066406$	< 0.1：停 stop (tol 0.1)
6	1.375	1.4375	1.40625	-0.022461	$a \leftarrow m$
7	1.40625	1.4375	1.421875	$+0.021729$	$b \leftarrow m$
8	1.40625	1.421875	**1.4140625**	-0.000427	< 0.01：停 stop (tol 0.01)

注意容差是对 $|f(m)|$ 而言的：tol $= 0.1$ 时 x 的实际误差 $|1.4375 - \sqrt{2}| = 0.023$；tol $= 0.01$ 时为 0.00015。两者的数值并不相同。

> The tolerance applies to $|f(m)|$: with tol $= 0.1$ the actual x error is $|1.4375 - \sqrt{2}| = 0.023$; with tol $= 0.01$ it is 0.00015. The two numbers are not the same.

局限 / Limitations

- 前提：初始区间 $[a, b]$ 必须变号。偶重根（如 x^2 在 0 处）不变号，二分法找不到。
  > Requirement: the initial interval $[a, b]$ must contain a sign change. A double root (such as x^2 at 0) has none, so bisection cannot find it.
- 收敛速度：线性收敛（每步误差界减半），比 Newton-Raphson 等方法慢。
  > Convergence speed: linear (the error bound halves each step), slower than methods such as Newton-Raphson.
- 异常处理：若 $f(a)$、$f(b)$ 不能括住根，方法会报错，说明所选区间不对（例：$[2, 4]$ 上 $x^2 - 2$ 不变号）。
  > Exception handling: if $f(a)$ and $f(b)$ do not bracket a root, the method raises an error showing the interval is wrong (e.g. $x^2 - 2$ has no sign change on $[2, 4]$).

考点速记 KEY POINTS

- tol 0.1 → 1.4375（5 步）；tol 0.01 → 1.4140625（8 步）。
  tol 0.1 → 1.4375 (5 steps); tol 0.01 → 1.4140625 (8).
- tol 限制的是 $|f|$，不是 $|x - x_r|$。
  tol limits $|f|$, not $|x - x_r|$.
- 必须变号；偶重根不行。
  Needs a sign change; double roots fail.
- 线性收敛：可靠但慢。
  Linear convergence: reliable but slow.

4 Newton-Raphson 法

4 Newton-Raphson method

- Newton-Raphson 法是求实值函数根的迭代方法。
  > The Newton-Raphson method is an iterative technique for finding roots of a real-valued function.
- 假设 $f(x)$ 光滑连续，根 x_r 未知；从初始猜测 x_0 出发，逐步改进。
  > It assumes a smooth, continuous $f(x)$ with an unknown root x_r, and starts from an initial guess x_0 that it improves iteratively.

线性近似 / Linear approximation

方法依赖 f 在初始猜测 x_0 附近的线性近似（一阶 Taylor 展开），令切线为零求下一点：

> The method relies on the linear approximation of f around the initial guess x_0 (first-order Taylor expansion); setting the tangent line to zero gives the next point:

切线近似 / tangent line

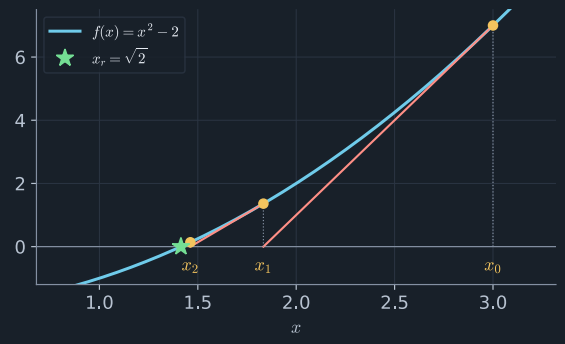
$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$
$$0 = f(x_0) + f'(x_0)(x_1 - x_0)$$

Newton 步 / Newton step

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

一般地 $x_{k+1} = x_k - f(x_k)/f'(x_k)$

/ in general $x_{k+1} = x_k - f(x_k)/f'(x_k)$



$x^2 - 2$ 从 $x_0 = 3$ 出发：每条红线是切线，与 x 轴的交点就是下一个猜测

/ $x^2 - 2$ from $x_0 = 3$: each red line is a tangent; where it meets the x-axis is the next guess

过程 / Process

1. 选初始猜测 x_0。
   > Start with an initial guess x_0.
2. 用公式得到更好的近似根。
   > Use the formula to find a better approximation of the root.
3. 重复，直到误差小于指定容差。
   > Repeat until the error is below the specified tolerance.

二次收敛 拓展 EXT / **Quadratic convergence**

在单根附近 $e_{k+1} \approx \left|\frac{f''(x_r)}{2f'(x_r)}\right| e_k^2$。对 $x^2 - 2$：系数 $= \frac{1}{2\sqrt{2}} = 0.354$，$e_0 = 0.086 \Rightarrow e_1 \approx 0.354 \times 0.086^2 = 0.0026$，实测 0.0025。

> Near a simple root, $e_{k+1} \approx \left|\frac{f''(x_r)}{2f'(x_r)}\right| e_k^2$. For $x^2 - 2$ the factor is $\frac{1}{2\sqrt{2}} = 0.354$, so $e_0 = 0.086 \Rightarrow e_1 \approx 0.354 \times 0.086^2 = 0.0026$; measured 0.0025.

考点速记 KEY POINTS

- $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$。
  $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$.
- 需要导数 f' 和初值 x_0。
  Needs f' and a guess x_0.
- 几何：切线与 x 轴的交点。
  Geometry: where the tangent meets the x-axis.
- 二次收敛：$e_{k+1} \propto e_k^2$。
  Quadratic: $e_{k+1} \propto e_k^2$.

4.2 Newton-Raphson 法的实现与局限

4.2 Implementation and limitations of the Newton-Raphson method

```python
import numpy as np

f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
```

```python
def my_newton(f, df, x0, tol):
    # output is an estimation of the root of f
    # using the Newton Raphson method
    # recursive implementation
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df,
                         x0 - f(x0)/df(x0), tol)
```

```python
estimate = my_newton(f, f_prime, 1.5, 1e-6)
print("estimate =", estimate)
print("sqrt(2) =", np.sqrt(2))
```

```
estimate = 1.4142135623746899
sqrt(2) = 1.4142135623730951
```

k	x_k	$\lvert x_k - \sqrt{2}\rvert$
0	1.5	8.6×10^{-2}
1	1.41666667	2.5×10^{-3}
2	1.41421569	2.1×10^{-6}
3	**1.4142135623746899**	1.6×10^{-12}

3 步就从 2 位有效数字到 12 位：误差指数大约每步翻倍。停止时 $|f(x_3)| = 4.5 \times 10^{-12} < 10^{-6}$。与二分法比：二分法要约 40 步才能达到 10^{-12}。

> Three steps take it from 2 correct digits to 12: the error exponent roughly doubles each step. It stops with $|f(x_3)| = 4.5 \times 10^{-12} < 10^{-6}$. For comparison, bisection needs about 40 steps to reach 10^{-12}.

主要局限 / Key limitations

- 对初值敏感：成功与否在很大程度上取决于 x_0 的选择。
  > Initial guess sensitivity: success depends heavily on the choice of x_0.
- 导数问题：若 $f'(x)$ 接近零，步长会很大且不稳定。
  > Derivative issues: if $f'(x)$ is close to zero, the method can take large, unstable steps.
- 收敛到哪个根：初值离根远时，可能收敛到另一个根，或不收敛。
  > Root convergence: from a guess far from the root it may converge to a different root, or not at all.

与其他方法比较 / Comparison with other methods

- 初值接近根时，Newton-Raphson 通常比二分法收敛快。
  > Newton-Raphson usually converges faster than bisection when the initial guess is close to the root.
- 但由于它很敏感，对行为复杂的函数不如二分法可靠。
  > Because of its sensitivity, it is less reliable for functions with complex behaviour.

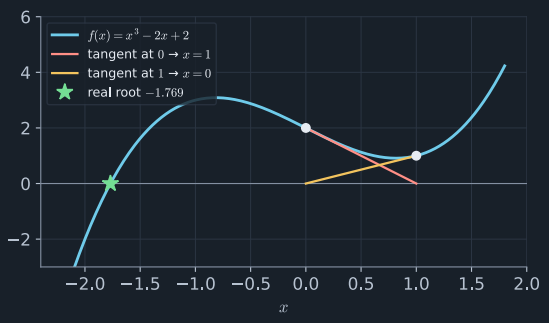
考点速记 KEY POINTS

- my_newton 递归：$|f(x_0)| < \text{tol}$ 时返回。
  my_newton recurses until $|f(x_0)| < \text{tol}$.
- $x_0 = 1.5$：3 步误差 1.6×10^{-12}。
  $x_0 = 1.5$: error 1.6×10^{-12} after 3 steps.
- $f' \approx 0 \to$ 步长巨大。
  $f' \approx 0 \to$ huge steps.
- 快但不保证收敛；二分法慢但保证。
  Fast but not guaranteed; bisection slow but sure.

4.3 Newton 法的失败情形与收敛对比 拓展 EXT

4.3 When Newton fails, and convergence compared

情形 Case	例子 Example	结果 Result
$f'(x_k)=0$	x^2-2，$x_0=0$	除以零，切线水平 · division by zero, flat tangent
f' 很小 · small f'	$x^3-100x^2-x+100$，$x_0=0$（$f'(0)=-1$）	一步跳到 $x_1=100$ · one step jumps to $x_1=100$
循环 · cycle	x^3-2x+2，$x_0=0$	$0\to1\to0\to1\cdots$ 永不收敛 · never converges
发散 · divergence	$1/x$（无根 · no root）	$x_{k+1}=2x_k$，越走越远 · runs away



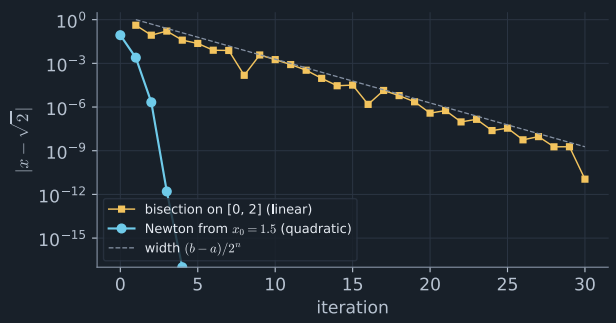
两条切线互相"送回"：$0\leftrightarrow1$

/ the two tangents send each other back: $0\leftrightarrow1$

循环的计算：$f(0)=2$、$f'(0)=-2$，$x_1=0-\frac{2}{-2}=1$；$f(1)=1$、$f'(1)=1$，$x_2=1-1=0$。改从 $x_0=-2$ 出发则收敛到唯一实根 -1.7693。对 $1/x$：$x_{k+1}=x_k-\frac{1/x_k}{-1/x_k^2}=2x_k$。

> The cycle: $f(0)=2$, $f'(0)=-2$, $x_1=0-\frac{2}{-2}=1$; $f(1)=1$, $f'(1)=1$, $x_2=1-1=0$. Starting from $x_0=-2$ instead converges to the only real root -1.7693. For $1/x$: $x_{k+1}=x_k-\frac{1/x_k}{-1/x_k^2}=2x_k$.

线性 vs 二次收敛 / Linear vs quadratic convergence



x^2-2：二分法误差沿虚线（每步 ×½）下降；Newton 4 步就到机器精度

/ x^2-2: the bisection error follows the dashed line (×½ per step); Newton hits machine precision in 4 steps

实用建议 / Practical advice

能括根时先括根：brentq（二分的可靠性 + 接近 Newton 的速度）是一维求根的好默认值。用 Newton 时设最大步数，并检查 $|x_{k+1}-x_k|$。

> If you can bracket the root, do: brentq (bisection's reliability with near-Newton speed) is a good default for 1-D roots. With Newton, cap the step count and check $|x_{k+1}-x_k|$.

考点速记 KEY POINTS

- $f'=0$：除以零。
  $f'=0$: division by zero.
- $1/x$ 的 Newton 步：$x\to2x$。
  Newton on $1/x$: $x\to2x$.
- x^3-2x+2 从 0 出发：$0\leftrightarrow1$ 循环。
  x^3-2x+2 from 0: cycles $0\leftrightarrow1$.
- 能括根就用 brentq。
  Bracket available: use brentq.

5 用 fsolve 求根

5 Using fsolve for root finding

例题所用函数：

The function used in the example:

$$f(x) = x^3 - 100x^2 - x + 100 = (x - 100)(x^2 - 1) = (x - 100)(x - 1)(x + 1)$$

```
from scipy.optimize import fsolve

f = lambda x: x**3 - 100*x**2 - x + 100
roots = fsolve(f, [2, 80])
print(roots)
```

```
[  1. 100.]
```

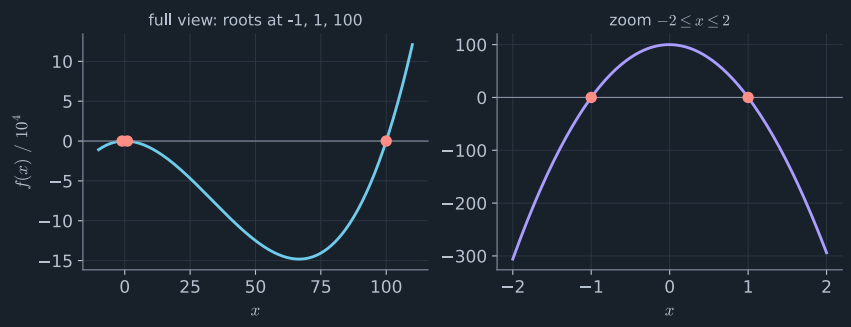
输出：[1., 100.]。初始猜测 $[2, 80]$ 使它收敛到 $x = 1$ 和 $x = 100$。传入长度为 2 的数组时，fsolve 把它当作两个互不相关的方程同时求解（f 逐元素计算），所以一次得到两个根。

Output: [1., 100.]. The initial guess $[2, 80]$ leads to $x = 1$ and $x = 100$. Given a length-2 array, fsolve treats it as two independent equations solved together (f is applied element-wise), so it returns two roots at once.

编者注 / Editorial note

幻灯片说初值 $[2, 80]$ 收敛到"该函数的两个根 $x = 1$ 和 $x = 100$"。三次多项式有三个根：因式分解显示还有 $x = -1$。fsolve 只返回离各初值"可达"的根，不会报告其余的根。

The slide says the guess $[2, 80]$ converges to "the two roots of the function: $x = 1$ and $x = 100$". The cubic has three roots: the factorisation shows $x = -1$ as well. fsolve returns only the roots reachable from each guess and says nothing about the others.



初值决定找到哪个根 拓展 EXT / The guess decides which root you get

初值 x_0	-2	0	2	50	80
fsolve 结果 result	-1	100	1	1	100

$x_0 = 0$ 离 ± 1 最近，却得到 100：$f(0) = 100$、$f'(0) = -1$，Newton 步 $x_1 = 0 - \frac{100}{-1} = 100$ 一步跳到远处的根。fsolve 有很多参数（xtol、maxfev、fprime、full_output 等），要读文档并理解它在做什么。

$x_0 = 0$ is closest to ± 1 yet gives 100: $f(0) = 100$, $f'(0) = -1$, so the Newton step $x_1 = 0 - \frac{100}{-1} = 100$ jumps straight to the far root. fsolve has many parameters (xtol, maxfev, fprime, full_output, …); read the documentation and understand what it is doing.

考点速记 KEY POINTS

- $f = (x - 100)(x - 1)(x + 1)$：三个根。
  $f = (x - 100)(x - 1)(x + 1)$: three roots.
- 初值不同 → 根不同，未必是最近的根。
  Different guess → different root, not always the nearest.
- fsolve(f, [2, 80]) → [1, 100]。
  fsolve(f, [2, 80]) → [1, 100].
- 求所有多项式根用 np.roots。
  All polynomial roots: np.roots.

公式总表
Formula sheet

求根问题 / root

$$f(x_r) = 0$$

误差度量 / error metrics

$$|f(x)| < \text{tol} \quad \text{or} \quad |x_{i+1} - x_i| < \text{tol}$$

介值定理 / intermediate value theorem

$$f(a)f(b) < 0 \Rightarrow \exists c \in (a,b),\ f(c) = 0$$

二分中点与步数 / bisection

$$m = \frac{a+b}{2}, \qquad n \geq \log_2 \frac{b-a}{\varepsilon}$$

Newton 步 / Newton step

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

收敛速度 / convergence rates

$$\text{bisection: } e_{k+1} \approx \tfrac{1}{2} e_k$$

$$\text{Newton: } e_{k+1} \approx \left| \frac{f''}{2f'} \right| e_k^2$$

方法 Method	需要 Needs	收敛 Converges	保证? Guaranteed?
二分法 Bisection	括根区间 · a bracket	线性 linear	是（有括根）· yes, given a bracket
Newton-Raphson	f'、好的初值 · f', a good guess	二次 quadratic	否 · no
fsolve	初值 · a guess	快 fast	否：查 ier · no: check ier
brentq	括根区间 · a bracket	超线性 superlinear	是 · yes

当前 SciPy 写法 / Current SciPy calls

```
from scipy.optimize import root_scalar, brentq, fsolve
sol = root_scalar(f, bracket=[0, 2], method='bisect', xtol=1e-10)
sol = root_scalar(f, x0=1.5, fprime=df, method='newton')
print(sol.root, sol.converged, sol.iterations)
r = brentq(f, 0, 2)              # bracketing and fast: a good default
x, info, ier, msg = fsolve(f, x0, full_output=True)
if ier != 1:                     # always check convergence
    print(msg)
```

考点速记 KEY POINTS

- 二分：可靠、慢；Newton：快、敏感。
  Bisection: sure but slow; Newton: fast but sensitive.
- fsolve：总要检查 ier。
  fsolve: always check ier.
- $n \geq \log_2 \frac{b-a}{\varepsilon}$。
  $n \geq \log_2 \frac{b-a}{\varepsilon}$.
- $|f|$ 与 $|\Delta x|$ 一起判断收敛。
  Judge convergence by $|f|$ and $|\Delta x|$ together.

01 / 6 二分法 · Bisection

手算二分法

Bisection by hand

对 $f(x)=x^3-x-2$ 在 $[1,2]$ 上做 5 步二分。写出每步的区间与中点。要保证中点误差 $\le 10^{-4}$，至少需要多少步?

> **Apply 5 bisection steps to $f(x)=x^3-x-2$ on $[1,2]$. Give each step's interval and midpoint. How many steps guarantee a midpoint error $\le 10^{-4}$?**

答案 Answer 1.53125 · 14

第 5 个中点 1.53125（真根 1.52138）；保证 10^{-4} 需 14 步。

> **The 5th midpoint is 1.53125 (true root 1.52138); 10^{-4} is guaranteed after 14 steps.**

检查括根：$f(1)=-2<0$，$f(2)=4>0$ ✓。

> Check the bracket: $f(1)=-2<0, f(2)=4>0$ ✓.

步 Step	$[a,b]$	m	$f(m)$	新区间 New interval
1	$[1,\ 2]$	1.5	-0.125	$[1.5,\ 2]$
2	$[1.5,\ 2]$	1.75	$+1.609$	$[1.5,\ 1.75]$
3	$[1.5,\ 1.75]$	1.625	$+0.666$	$[1.5,\ 1.625]$
4	$[1.5,\ 1.625]$	1.5625	$+0.252$	$[1.5,\ 1.5625]$
5	$[1.5,\ 1.5625]$	**1.53125**	$+0.059$	$[1.5,\ 1.53125]$

$$|m_n-x_r|\le\frac{b-a}{2^n}=\frac{1}{2^n}\le 10^{-4}\iff 2^n\ge 10^4\iff n\ge\log_2 10^4=13.3\Rightarrow n=14$$

验证：第 5 个中点误差 $|1.53125-1.52138|=0.0099\le\frac{1}{2^5}=0.031$ ✓。

> Check: the 5th midpoint's error $|1.53125-1.52138|=0.0099\le\frac{1}{2^5}=0.031$ ✓.

考点速记 KEY POINTS

- 先验证 $f(a)f(b)<0$。
  First verify $f(a)f(b)<0$.
- 误差界 $(b-a)/2^n$。
  Error bound $(b-a)/2^n$.
- $f(m)$ 与 $f(a)$ 同号 $\to a\leftarrow m$。
  $f(m)$ has the sign of $f(a)\to a\leftarrow m$.
- 10^{-4} 精度：14 步。
  10^{-4} accuracy: 14 steps.

02 / 6 Newton-Raphson

手算 Newton 迭代

Newton iterations by hand

用 Newton-Raphson 法求 $\cos x = x$ 的根，$x_0 = 1$。写出 Newton 公式并计算 x_1, x_2, x_3，说明收敛的阶。

> **Use Newton-Raphson to solve $\cos x = x$ from $x_0 = 1$. Write the Newton formula, compute x_1, x_2, x_3, and state the order of convergence.**

答案 Answer 0.739085

$x_1 = 0.750364$，$x_2 = 0.739113$，$x_3 = 0.739085$；**误差** $10^{-2} \to 10^{-5} \to 10^{-10}$：**二次收敛。**

> $x_1 = 0.750364$, $x_2 = 0.739113$, $x_3 = 0.739085$; **errors** $10^{-2} \to 10^{-5} \to 10^{-10}$: **quadratic.**

推导 / Derivation

$$f(x) = \cos x - x, \qquad f'(x) = -\sin x - 1, \qquad x_{k+1} = x_k - \frac{\cos x_k - x_k}{-\sin x_k - 1}$$

$$x_1 = 1 - \frac{0.540302 - 1}{-0.841471 - 1} = 1 - \frac{-0.459698}{-1.841471} = 1 - 0.249636 = 0.750364$$

计算器使用弧度制 / calculator in radians

k	x_k	$\lvert x_k - x_r \rvert$
1	0.750364	1.1×10^{-2}
2	0.739113	2.8×10^{-5}
3	**0.739085**	1.7×10^{-10}

误差的指数 $-2 \to -5 \to -10$ 约每步翻倍，符合 $e_{k+1} \approx C e_k^2$；这里 $C = \left|\frac{f''}{2f'}\right| = \frac{|\cos x_r|}{2(1+\sin x_r)} = 0.22$，$0.22 \times (2.8 \times 10^{-5})^2 = 1.7 \times 10^{-10}$ ✓。

> The error exponent goes $-2 \to -5 \to -10$, roughly doubling each step, as $e_{k+1} \approx C e_k^2$ predicts; here $C = \left|\frac{f''}{2f'}\right| = \frac{|\cos x_r|}{2(1+\sin x_r)} = 0.22$, and $0.22 \times (2.8 \times 10^{-5})^2 = 1.7 \times 10^{-10}$ ✓.

考点速记 KEY POINTS

- 先写出 f 和 f'，注意符号。
  Write f and f' first; mind the signs.
- $f' = -\sin x - 1$。
  $f' = -\sin x - 1$.
- 指数翻倍 = 二次收敛。
  Exponent doubling = quadratic convergence.
- 弧度制！
  Radians!

03 / 6 容差 · Tolerance

解读 fsolve 对 $1/x$ 的输出

Interpreting fsolve's output for $1/x$

fsolve 对 $f(x) = 1/x$（初值 -2）返回 $r = -3.52 \times 10^{83}$，$f(r) = -2.84 \times 10^{-84}$，并给出消息 "The number of calls to function has reached maxfev = 400." 正确的结论是?

For $f(x) = 1/x$ (guess -2), fsolve returns $r = -3.52 \times 10^{83}$ with $f(r) = -2.84 \times 10^{-84}$ and the message "The number of calls to function has reached maxfev = 400." What is the correct conclusion?

A r 是根，因为 $f(r) \approx 0$。
r is a root because $f(r) \approx 0$.

B 求解失败：$1/x$ 没有根，$|f|$ 小只是因为 $|x|$ 极大。 ✓ 正确 Correct
The solve failed: $1/x$ has no root; $|f|$ is small only because $|x|$ is huge.

C 把 maxfev 调大就能得到真正的根。
Raising maxfev will give the true root.

D 应报告根在 $-\infty$。
Report the root as $-\infty$.

答案 Answer B

消息（ier = 2）说明失败；$1/x = 0$ 无解。
The message (ier = 2) signals failure; $1/x = 0$ has no solution.

$$\frac{1}{x} = 0 \text{ has no solution;} \qquad |f(x)| = \frac{1}{|x|} < \text{tol} \iff |x| > \frac{1}{\text{tol}}$$

干扰项为什么错 / Why the distractors are wrong

A 只看残差，正是幻灯片 4 警告的"假收敛"；代回 $f(r) \neq 0$ 而且 r 不合理。
Judging by the residual alone is the false convergence slide 4 warns about; $f(r) \neq 0$ and r is implausible.

C 没有根可找，多算只会让 $|x|$ 更大。
There is no root to find; more calls only make $|x|$ larger.

D $-\infty$ 不是实数，不是根；$\lim f = 0$ 不等于 f 取到 0。
$-\infty$ is not a real number, so not a root; $\lim f = 0$ does not mean f reaches 0.

考点速记 KEY POINTS

- 先读 mesg / ier。
  Read mesg / ier first.
- 检查 r 是否合理。
  Check whether r is plausible.
- 渐近线 $f \to 0$ 会骗过 $|f|$ 判据。
  An asymptote $f \to 0$ fools the $|f|$ test.
- 变号检查：$1/x$ 跨 0 变号但不连续。
  Sign test: $1/x$ flips at 0 but is discontinuous.

04 / 6 二分步数 · Bisection steps

二分法需要多少步

How many bisection steps?

用二分法在 $[0, 2]$ 上求 $\sqrt{2}$。要保证中点与真根的距离不超过 10^{-6}，至少要计算多少个中点?

> **Bisection on $[0, 2]$ is used to find $\sqrt{2}$. How many midpoints must be computed to guarantee the midpoint is within 10^{-6} of the true root?**

A 6 个

> 6

B 21 个 ✓ 正确 Correct

> 21

C 约 100 万个

> about 1,000,000

D 3 个

> 3

答案 Answer B

$\frac{2}{2^n} \le 10^{-6} \Rightarrow n \ge \log_2(2 \times 10^6) = 20.9$，**取 21。**

> $\frac{2}{2^n} \le 10^{-6} \Rightarrow n \ge \log_2(2 \times 10^6) = 20.9$, **so 21.**

$$|m_n - x_r| \le \frac{b-a}{2^n} = \frac{2}{2^n} \le 10^{-6} \iff 2^n \ge 2 \times 10^6 \iff n \ge 20.93 \Rightarrow n = 21$$

每步误差界减半，所以每 $\log_2 10 \approx 3.3$ 步多一位有效数字：6 位小数约 20 步。对比 Newton（第 4.2 节）：从 1.5 出发 3 步就到 10^{-12}。

> The bound halves each step, so each extra correct digit costs $\log_2 10 \approx 3.3$ steps: about 20 steps for 6 decimals. Compare Newton (Section 4.2): 3 steps from 1.5 reach 10^{-12}.

干扰项为什么错 / Why the distractors are wrong

A 把"每步一位小数"当成规律；实际每步只得约 0.3 位。

> Assumes one decimal digit per step; each step gives only about 0.3 digits.

C 把区间按等长小步扫描的思路；二分是几何减半，不是线性扫描。

> Thinks of scanning in equal small steps; bisection halves geometrically.

D 是 Newton 法的量级；二分法做不到。

> That is Newton's order of magnitude; bisection cannot do it.

考点速记 KEY POINTS

- $n \ge \log_2 \frac{b-a}{\varepsilon}$。
  $n \ge \log_2 \frac{b-a}{\varepsilon}$.
- 步数与 f 的形状无关。
  The count is independent of f's shape.
- 每位有效数字 ≈ 3.3 步。
  ≈ 3.3 steps per digit.
- $\log_2(2 \times 10^6) = 20.9$，向上取整。
  $\log_2(2 \times 10^6) = 20.9$: round up.

05 / 6 Newton 失败 · Newton failure (EXT)

Newton 法一定收敛吗

Does Newton's method always converge?

对 $f(x) = x^3 - 2x + 2$ 从 $x_0 = 0$ 出发做 Newton 迭代。它会收敛到根吗?

Run Newton's method on $f(x) = x^3 - 2x + 2$ from $x_0 = 0$. Does it converge to a root?

答案 Answer 否 · No

迭代在 0 与 1 之间无限循环；从 $x_0 = -2$ 出发才收敛到 -1.7693。

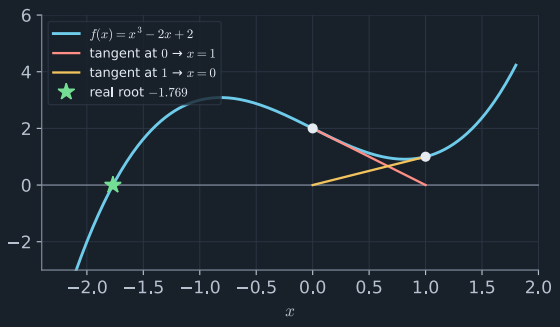
The iterates cycle between 0 and 1 forever; from $x_0 = -2$ it converges to -1.7693.

推导 / Derivation

$$f'(x) = 3x^2 - 2$$

$$x_1 = 0 - \frac{f(0)}{f'(0)} = 0 - \frac{2}{-2} = 1$$

$$x_2 = 1 - \frac{f(1)}{f'(1)} = 1 - \frac{1}{1} = 0 \quad \Rightarrow \quad 0, 1, 0, 1, \ldots$$



这是幻灯片"可能不收敛"的具体例子。用 my_newton 跑这个例子会无限递归（$|f|$ 在 2 与 1 之间交替，永远不小于 tol），最终 RecursionError。补救：换初值、设最大步数，或先用二分法（$f(-2) = -2 < 0$，$f(-1) = 3 > 0$）括住根再切换到 Newton。

This is a concrete case of the slide's "may fail to converge". Running my_newton here recurses forever ($|f|$ alternates between 2 and 1 and never drops below tol) until a RecursionError. Remedies: change the guess, cap the steps, or first bracket the root with bisection ($f(-2) = -2 < 0$, $f(-1) = 3 > 0$) and then switch to Newton.

考点速记 KEY POINTS

- Newton 可能循环，不收敛。
  Newton can cycle without converging.
- $x^3 - 2x + 2$：$0 \to 1 \to 0$。
  $x^3 - 2x + 2$: $0 \to 1 \to 0$.
- 一定要设最大迭代次数。
  Always cap the iterations.
- 先括根再 Newton 更稳。
  Bracket first, then Newton, is safer.

06 / 6 fsolve 与初值 · fsolve and guesses

fsolve 找到哪个根

Which root does fsolve find?

$f(x) = x^3 - 100x^2 - x + 100$。它有几个实根？从初值 $x_0 = -2,\ 0,\ 2$ 运行 fsolve，分别得到什么？为什么 $x_0 = 0$ 的结果出人意料?

> $f(x) = x^3 - 100x^2 - x + 100$. **How many real roots does it have? What does fsolve return from $x_0 = -2,\ 0,\ 2$? Why is the result for $x_0 = 0$ surprising?**

答案 Answer 3 roots: –1, 100, 1

三个根 $-1, 1, 100$；初值 $-2 \to -1$，$0 \to 100$，$2 \to 1$。

> **Three roots $-1, 1, 100$; guesses $-2 \to -1$, $0 \to 100$, $2 \to 1$.**

推导 / Derivation

$$f(x) = x^2(x-100) - (x-100) = (x-100)(x^2-1) = (x-100)(x-1)(x+1)$$

$$x_0 = 0: \quad f(0) = 100,\ f'(0) = 3(0)^2 - 200(0) - 1 = -1 \ \Rightarrow\ x_1 = 0 - \frac{100}{-1} = 100$$

$|f'|$ 小、$|f|$ 大 → 一步跳到远处的根 / small $|f'|$, large $|f|$ → one step to the far root

$x_0 = 0$ 离 ± 1 都只有 1，却得到 100：这正是幻灯片所说"导数接近零时步长很大"以及"可能收敛到另一个根"。幻灯片例子用 $[2, 80]$ 只得到 1 和 100，所以"该函数的两个根"漏掉了 -1。要列出多项式的全部根，用 np.roots([1, –100, –1, 100])，得到 $100,\ -1,\ 1$。

> $x_0 = 0$ is just 1 away from ± 1 yet lands on 100: exactly the slide's "large steps when the derivative is near zero" and "may converge to a different root". The slide's guess $[2, 80]$ yields only 1 and 100, so "the two roots of the function" misses -1. To list every root of a polynomial use np.roots([1, –100, –1, 100]), which gives $100,\ -1,\ 1$.

考点速记 KEY POINTS

- 三次多项式最多 3 个实根；先试因式分解。
  A cubic has up to 3 real roots; try factoring first.
- 离得近 ≠ 收敛到它。
  Nearest root ≠ the one you get.
- 初值 0 → 100（第一步就跳过去）。
  Guess 0 → 100 (the first step jumps).
- 多项式全部根：np.roots。
  All polynomial roots: np.roots.

答案速查与来源

Answer key and sources

题号 Q	答案 Ans.	要点 Key idea
1	**1.53125 · 14**	误差界 $(b-a)/2^n$ · error bound $(b-a)/2^n$
2	**0.739085**	误差 $10^{-2} \to 10^{-5} \to 10^{-10}$：二次收敛 · quadratic
3	**B**	$\lvert f\rvert$ 小 ≠ 根；读 ier/mesg · small $\lvert f\rvert$ ≠ root; read ier/mesg
4	**B**	$n \geq \log_2(2\times 10^6) = 20.9 \to 21$
5	**否 No ***	$0 \leftrightarrow 1$ 循环 · cycles $0 \leftrightarrow 1$
6	$-1,\ 100,\ 1$ *	初值 0 时 $f'(0) = -1$，一步到 100 · guess 0: $f'(0) = -1$, jumps to 100

带 * 的题超出幻灯片。幻灯片第 10 页"两个根"漏掉了 $x = -1$，见第 5 节编者注。

> Starred items go beyond the slides. Slide 10's "two roots" misses $x = -1$; see the Section 5 editorial note.

来源 / Sources

1. PHYB57 C19 lecture slides, "Root Finding" (PHYB57_C19.pdf, 10 slides)
   用户上传 · uploaded by the user
2. Q. Kong, T. Siauw, A. Bayen, Python Programming and Numerical Methods, Ch. 19 Root Finding
   https://pythonnumericalmethods.berkeley.edu/notebooks/chapter19.00-Root-Finding.html
3. Same book, §19.3 Bisection Method · §19.4 Newton-Raphson Method
   https://pythonnumericalmethods.berkeley.edu/notebooks/chapter19.04-Newton-Raphson-Method.html
4. SciPy documentation, scipy.optimize.fsolve
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.fsolve.html
5. SciPy documentation, root_scalar · brentq
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.root_scalar.html

图均按幻灯片代码和例子用 Matplotlib 重新绘制；数值由 Python（SciPy 1.18.1）复算，包括幻灯片 2、7、9、10 的全部输出。

> Figures were redrawn with Matplotlib from the slides' code and examples; numbers were recomputed in Python (SciPy 1.18.1), including every output on slides 2, 7, 9 and 10.

考点速记 KEY POINTS

- 二分：括根、减半、线性。
  Bisection: bracket, halve, linear.
- 容差：$\lvert f\rvert$ 与 $\lvert\Delta x\rvert$ 都要看。
  Tolerance: watch both $\lvert f\rvert$ and $\lvert\Delta x\rvert$.
- Newton：切线、二次、看初值。
  Newton: tangent, quadratic, guess-dependent.
- $x^3 - 100x^2 - x + 100$ 有 3 个根。
  $x^3 - 100x^2 - x + 100$ has 3 roots.